

小红书前端面经

小红书前端面试一面

题目

1. 实现数组扁平化的多种方法
2. 解释闭包及其常见应用场景
3. CSS选择器优先级计算规则
4. 手写Promise.all
5. 解释浏览器事件循环机制

小红书前端面试二面

题目

1. 虚拟DOM diff算法具体实现原理
2. 前端性能优化方案（至少5种）
3. 实现函数柯里化的两种方式
4. 解释HTTP2新特性
5. React Fiber架构设计思想
6. Webpack热更新原理
7. 如何实现0.5px边框

小红书前端面试三面

题目

1. 设计前端错误监控系统
2. 实现大文件分片上传/断点续传
3. 解释Vue3响应式原理
4. 前端安全防护策略（XSS/CSRF）
5. 实现LRU缓存算法
6. 微前端架构落地方案

7. Web Workers通信原理
8. Node.js事件循环与浏览器差异
9. 首屏加载时间优化方案
10. 解释BFC及其应用场景

HR面试

题目

1. 为什么选择小红书?
 2. 如何处理团队意见分歧?
 3. 最近学习的非技术类书籍
 4. 职业发展规划
-

参考答案

小红书前端面试一面

1. 实现数组扁平化的多种方法

- 方法 1: 使用 `Array.prototype.flat()`

▼ 代码块

```
1  const arr = [1, [2, 3, [4, 5]]];
2  const flatArr = arr.flat(Infinity); // 扁平化到任意深度
```

- 方法 2: 使用递归

▼ 代码块

```
1  function flatten(arr) {
2    let result = [];
3    arr.forEach(item => {
4      if (Array.isArray(item)) {
5        result = result.concat(flatten(item));
6      } else {
7        result.push(item);
8      }
9    });
10   return result;
11 }
```

- 方法 3: 使用 `reduce`

▼ 代码块

```
1 function flatten(arr) {
2   return arr.reduce((acc, item) => {
3     return Array.isArray(item) ? acc.concat(flatten(item)) :
      acc.concat(item);
4   }, []);
5 }
```

- 方法 4: 使用 `JSON.parse` 和 `JSON.stringify`

▼ 代码块

```
1 const arr = [1, [2, 3, [4, 5]]];
2 const flatArr = JSON.parse('[' + JSON.stringify(arr).replace(/\[|\]/g, '')
  + ']');
```

2. 解释闭包及其常见应用场景

- 闭包是指函数和声明该函数的词法环境绑定在一起，形成一个“包裹”函数及其外部变量的作用域链。

- 应用场景：

- i. 私有变量：通过闭包实现数据封装，避免外部直接访问。

▼ 代码块

```
1 function createCounter() {
2   let count = 0;
3   return {
4     increment: () => count++,
5     decrement: () => count--,
6     getCount: () => count
7   };
8 }
9 const counter = createCounter();
10 console.log(counter.getCount()); // 0
11 counter.increment();
12 console.log(counter.getCount()); // 1
```

- i. 函数工厂：根据不同需求创建不同的函数。

ii. 回调函数和事件处理：事件处理中常常使用闭包来保持对外部变量的引用。

3. CSS选择器优先级计算规则

- 优先级计算顺序：

- i. 内联样式 > ID选择器 > 类选择器 > 元素选择器

- ii. 重要声明（`!important`）总是优先。

- iii. 计算优先级时，ID选择器的优先级为 100、类选择器为 10、元素选择器为 1。

4. 手写Promise.all

▼ 代码块

```
1  function myPromiseAll(promises) {
2    return new Promise((resolve, reject) => {
3      let result = [];
4      let completed = 0;
5      promises.forEach((promise, index) => {
6        Promise.resolve(promise).then(value => {
7          result[index] = value;
8          completed++;
9          if (completed === promises.length) {
10             resolve(result);
11           }
12         }).catch(reject);
13       });
14     });
15   }
```

1. 解释浏览器事件循环机制

- 事件循环是浏览器中处理异步操作的机制。浏览器的主线程通过事件循环不断从任务队列中取出任务并执行。
- 执行顺序：
 - i. 执行同步代码（主线程）。
 - ii. 执行微任务（如 `Promise` 的回调）。
 - iii. 执行宏任务（如 `setTimeout`、I/O等）。
- 微任务优先级高于宏任务。

小红书前端面试二面

1. 虚拟DOM diff算法具体实现原理

- **虚拟DOM**是 React 等库用来高效更新视图的技术。其基本思路是将实际的 DOM 树与虚拟 DOM 树进行比较，找到差异并进行最小的更新。
- **diff算法**:
 - i. 比较节点类型：如果不同，直接替换。
 - ii. 比较节点的属性和文本内容。
 - iii. 比较子节点，通过递归比较。
 - iv. 使用**双指针**（同时遍历旧树和新树）来减少计算量。

2. 前端性能优化方案（至少5种）

- a. **懒加载**：只加载当前视口内的资源。
- b. **资源压缩与合并**：压缩 JavaScript、CSS 和图片等资源，减少文件大小和请求次数。
- c. **代码分割**：使用 Webpack 等工具实现按需加载，减少初始加载的资源大小。
- d. **缓存优化**：合理设置缓存策略，减少重复请求。
- e. **图片优化**：使用 WebP 格式、懒加载和响应式图片等手段。

3. 实现函数柯里化的两种方式

- **方法 1: 使用闭包**

▼ 代码块

```
1  function curry(fn) {
2    return function(...args) {
3      if (args.length >= fn.length) {
4        return fn(...args);
5      } else {
6        return function(...newArgs) {
7          return curry(fn)(...args, ...newArgs);
8        };
9      }
10   };
11 }
```

- **方法 2: 使用递归**

▼ 代码块

```
1  function curry(fn, arity = fn.length) {
2    return function(...args) {
3      if (args.length >= arity) {
4        return fn(...args);
5      }
6    }
7  }
```

```
6     return curry(fn.bind(null, ...args), arity - args.length);
7   };
8 }
```

4. 解释HTTP2新特性

- **多路复用**：允许多个请求/响应在同一个连接上并行传输。
- **头部压缩**：通过HPACK压缩HTTP头部，减少传输数据量。
- **服务器推送**：服务器可以主动推送资源到客户端，而不是等客户端请求。
- **流优先级和依赖**：可以设置流的优先级，优化请求的顺序和资源加载。

5. React Fiber架构设计思想

- React Fiber 是 React 的一种新的协调算法，旨在提高大规模应用的渲染性能。
- **核心思想**：
 - i. **增量渲染**：将渲染任务分解为多个小任务，允许中断和恢复，避免阻塞。
 - ii. **优先级调度**：根据任务的优先级动态决定何时执行。
 - iii. **Fiber节点**：每个渲染更新都是一个Fiber节点，具有执行状态、优先级等信息。

6. Webpack热更新原理

- **热模块替换（HMR）** 允许在不刷新页面的情况下更新模块。
- **原理**：
 - i. Webpack 监控文件的变化。
 - ii. 当发生变化时，Webpack 会将更新的模块发送到客户端。
 - iii. 客户端接收更新并通过 HMR API 替换更新的模块。

7. 如何实现0.5px边框

- 使用 **transform** 或 **svg** 来解决高DPI设备上0.5px边框模糊的问题。

▼ 代码块

```
1  .border {
2    border: 1px solid transparent;
3    background: linear-gradient(to right, transparent 49.5%, black 50%,
4    transparent 50.5%);
5    background-size: 100% 1px;
6  }
```

小红书前端面试三面

1. 设计前端错误监控系统

- 系统设计：

- i. 捕获前端错误：通过 `window.onerror`、`Promise.reject` 捕获同步和异步错误。
- ii. 错误上报：将错误信息上报至后端服务器，附带堆栈信息、浏览器版本等。
- iii. 数据存储和分析：后端存储错误日志，并提供分析工具。
- iv. 实时告警：系统可根据错误类型和频率实时发送告警通知。

2. 实现大文件分片上传/断点续传

- 实现思路：

- i. 文件分片：将文件分割成多个小片，每片发送一个 HTTP 请求。
- ii. 断点续传：记录每个片的上传状态，未成功上传的片可以重新上传。
- iii. 合并文件：服务器端合并所有上传的片段。

3. 解释Vue3响应式原理

- Vue3响应式使用了Proxy：

- i. 代理数据：通过Proxy对象代理数据对象的getter和setter方法。
- ii. 依赖收集：当访问某个数据时，会触发getter，收集依赖。
- iii. 视图更新：当数据变化时，触发setter，通知视图更新。

4. 前端安全防护策略（XSS/CSRF）

- XSS防护：

- i. 输入验证：对用户输入进行严格的过滤和转义。
- ii. 内容安全策略（CSP）：通过CSP限制可执行的脚本源。

- CSRF防护：

- i. 使用Token：在请求中加入CSRF Token进行校验。
- ii. SameSite Cookies：限制跨站请求带上Cookie。

5. 实现LRU缓存算法

- 使用双向链表和哈希表实现LRU缓存：

▼ 代码块

```
1  class LRUCache {
2    constructor(capacity) {
3      this.capacity = capacity;
4      this.cache = new Map();
5    }
6
7    get(key) {
8      if (!this.cache.has(key)) return -1;
```

```
9     const value = this.cache.get(key);
10    this.cache.delete(key);
11    this.cache.set(key, value);
12    return value;
13  }
14
15  put(key, value) {
16    if (this.cache.has(key)) this.cache.delete(key);
17    else if (this.cache.size == this.capacity)
18      this.cache.delete(this.cache.keys().next().value);
19    this.cache.set(key, value);
20  }
```

6. 微前端架构落地方案

- 使用微前端将一个应用拆解为多个子应用，每个子应用拥有自己的独立开发和部署周期。
- 采用框架间通信协议（如 `custom events`、`postMessage`）和公共样式管理（如 CSS Module）来实现不同微应用之间的隔离和通信。

7. Web Workers通信原理

- **Web Workers** 是浏览器中的线程，用于执行并行任务，避免阻塞主线程。
- 通信：
 - i. **postMessage**：主线程和Worker线程之间通过 `postMessage` 方法传递消息。
 - ii. **onmessage**：监听消息，处理数据。

8. Node.js事件循环与浏览器差异

- **Node.js事件循环**：是基于非阻塞I/O的事件驱动模型，Node.js的事件循环主要处理I/O任务、定时器和回调函数。
- **浏览器事件循环**：浏览器事件循环处理的主要是用户输入、渲染和UI更新。

9. 首屏加载时间优化方案

- 减少HTTP请求数**：合并文件、使用图标字体等。
- CDN加速**：使用CDN缓存静态资源，提高加载速度。
- 预加载**：使用 `<link rel="preload">` 预加载关键资源。
- 异步加载JS**：使用 `async` 或 `defer` 属性加载非关键JS。

10. 解释BFC及其应用场景

- **BFC**（块级格式化上下文）是Web布局中的一个概念，主要影响块级元素的布局方式。
- **应用场景**：解决浮动元素的问题，清除浮动，防止布局重叠。

